

Punjab University College of Information Technology
University of the Punjab, Lahore



Subject: Advanced Database Management Systems (ADBMS)

Project Report

Hospital Management System

Submitted By:

Ahmad Fayyaz	BCSF24M041
Ali Hamza	BCSF24M039
Muhammad Kazim	BCSF24M063

Instructor: Mam Saba Inam

Academic Year: 2025-26

1. Introduction

This report presents a Hospital Management System developed as a project for the subject Advanced Database Management Systems (ADBMS). The system is built using MongoDB, a NoSQL document-oriented database, and demonstrates how a hospital can manage its core data efficiently.

The system covers four main areas of hospital operations: patient records, doctor information, appointment scheduling, and medicine inventory. The project fulfills all the required tasks including collection creation, data insertion, CRUD operations, filtering and sorting queries, and aggregation pipelines.

2. System Overview

The Hospital Management System is designed to represent a realistic hospital environment. Hospital staff can use this system to record patient details, assign appointments with doctors, track which medicines were prescribed, and monitor medicine inventory levels.

The database used is named hospitalDB. It is created in MongoDB and contains four collections. All queries were written and executed using the MongoDB for VS Code extension.

3. Database Design

3.1 Collections

The hospitalDB database contains the following four collections:

Collection Name	Documents	Purpose
patients	14	Stores patient personal and medical details
doctors	14	Stores doctor profiles and consultation fees
appointments	14	Links patients, doctors, and prescribed medicines
medicines	14	Stores medicine inventory and supplier details

3.2 Fields in Each Collection

patients:

- patient_id, name, age, gender, blood_group, contact, city, disease

doctors:

- doctor_id, name, specialization, experience_years, fee, available

appointments:

- appointment_id, patient_id, doctor_id, medicine_id, date, status, notes

medicines:

- medicine_id, name, category, price, stock, supplier

3.3 Relationships

The appointments collection connects all four collections together. Each appointment document contains a patient_id, a doctor_id, and a medicine_id. This creates a logical link between a patient, the doctor they visited, and the medicine that was prescribed during that visit. This design follows MongoDB's document model where references are stored as field values rather than using foreign keys as in relational databases.

4. CRUD Operations

4.1 Insert

All four collections were first cleared using the drop() method to ensure no previous data remained. Sample data was then inserted using the insertMany() method. Each collection received 14 documents containing meaningful and realistic hospital data.

4.2 Find

Read operations were performed using the find() method. Queries were written to retrieve all records from a collection as well as filtered records based on specific conditions. Projection was also used to display only selected fields in the output.

4.3 Update

Update operations were carried out using updateOne() and updateMany(). The \$set operator was used to change specific field values such as a patient's disease or an appointment's status. The \$inc operator was used to increment numerical values such as medicine stock quantity.

4.4 Delete

Delete operations were performed using deleteOne() and deleteMany(). A specific appointment was deleted by its ID using deleteOne(). Medicines with stock below a certain level were removed using deleteMany() with a comparison filter.

5. Filter and Sort Queries

Filter queries were applied using MongoDB comparison operators to retrieve specific subsets of data. The following examples were implemented:

- All male patients from the patients collection
- Patients belonging to a specific city such as Multan
- Doctors who are currently available for appointments
- Doctors with a consultation fee below 1500
- Medicines with stock quantity below 160
- All confirmed appointments

Sort queries were applied using the sort() method. Results were sorted in both ascending (1) and descending (-1) order. Examples include sorting patients by age, sorting doctors by fee from highest to lowest, and sorting medicines by price from lowest to highest.

Filter and sort were also combined in single queries. For example, retrieving patients above the age of 40 and sorting them by age in ascending order, or listing available doctors sorted by fee in ascending order.

6. Aggregation Queries

Two aggregation queries were implemented using the aggregate() method. Aggregation in MongoDB allows data to be processed through a series of stages called a pipeline. Each stage transforms the data before passing it to the next stage.

6.1 Total Appointments Per Doctor

This query groups the appointments collection by doctor_id and counts the total number of appointments assigned to each doctor. The \$group stage is used for grouping and the \$sum operator is used for counting. The result is then sorted in descending order using the \$sort stage to show which doctor has the highest number of appointments.

6.2 Patient Count Per Disease

This query groups the patients collection by disease and counts how many patients are affected by each condition. This helps identify the most common diseases being treated in the hospital. The result is sorted in descending order by patient count.

7. Conclusion

This project successfully demonstrates the design and practical implementation of a Hospital Management System using MongoDB. All required tasks have been completed including the creation of four collections, insertion of meaningful sample data, CRUD operations, filter and sort queries, and aggregation pipelines.

The appointments collection plays a central role in connecting all other collections by holding references to patient, doctor, and medicine records. This design reflects how data relationships can be managed in a NoSQL database without the use of traditional relational joins.

The project was developed and tested using the Mongosh CLI and all the screenshots are attached with.